

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки та комп'ютерних технологій

ЗВІТ

до лабораторної роботи №1

«Організація професійного Python-проєкту та Git»

з курсу «Професійний Python»

Варіант №2 — «Система обліку книг»

Виконав(ла): Рибак Олександра

Група: ФеП-32

Перевірів(ла): Куньо І.М.

Дата: 14.09.2026

Львів — 2026

1. Мета роботи

Ознайомитися з основними принципами організації професійного Python-проєкту, навчитися створювати ізольоване середовище розробки, формувати структуру за принципом `src-layout`, працювати з модулями та пакетами Python, налаштовувати конфігураційний файл `pyproject.toml`, використовувати розподілену систему контролю версій Git і оформлювати базову проєктну документацію. Практично реалізувати консольний застосунок за індивідуальним варіантом №2 — «Система обліку книг».

2. Індивідуальний варіант і постановка задачі

Варіант №2 — «Система обліку книг».

Для кожної книги зберігати:

- назву (`title`);
- автора (`author`);
- рік видання (`year`);
- кількість сторінок (`pages`).

Необхідно реалізувати бізнес-функціонал:

- додавання нової книги (`add_book`);
- пошук книг за автором (`find_books_by_author`);
- фільтрацію книг за роком видання (`filter_by_year`);
- визначення найбільшої книги за кількістю сторінок (`find_largest_book`);
- обчислення середньої кількості сторінок у колекції (`calculate_average_pages`).

Проєкт повинен мати структуру `src-layout`, містити щонайменше три Python-модулі, модель даних, окремий модуль із бізнес-логікою, точку входу `main.py`, анотації типів (`type hints`), файли `README.md`, `.gitignore`, `pyproject.toml` та коректно сформовану історію комітів у репозиторії Git.

3. Структура проєкту

`book_manager/`

└── `.gitignore`

└── `pyproject.toml`

```
|— README.md
|— src/
    |— book_manager/
        |— __init__.py
        |— main.py
        |— models.py
        |— services.py
```

Каталог `src/book_manager` містить вихідний код пакета: модуль `models.py` описує структуру даних `Book`, `services.py` інкапсулює функції бізнес-логіки та обробки колекцій, `main.py` виступає точкою входу, а `__init__.py` ініціалізує пакет і визначає його версію.

4. Опис створених модулів

4.1. `models.py`

Для представлення сутності книги використано дата-клас `Book` із полями `title`, `author`, `year` та `pages`. Додаткова властивість `short_info` формує стислий рядок із метаданими книги для виведення.

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class Book:
```

```
    title: str
```

```
    author: str
```

```
    year: int
```

```
    pages: int
```

```
@property
```

```
    def short_info(self) -> str:
```

```
return f'''{self.title}' by {self.author} ({self.year})''
```

4.2. services.py

Модуль реалізує чисту бізнес-логіку програми: функції `add_book()`, `find_books_by_author()`, `filter_by_year()`, `find_largest_book()` та `calculate_average_pages()`. Усі операції ізольовані від консольного інтерфейсу та працюють виключно з типізованими аргументами.

```
from book_manager.models import Book
```

```
def add_book(books: list[Book], book: Book) -> None:
```

```
    books.append(book)
```

```
def find_books_by_author(books: list[Book], author: str) -> list[Book]:
```

```
    normalized_author = author.strip().casefold()
```

```
    return [
```

```
        book for book in books
```

```
        if normalized_author in book.author.casefold()
```

```
    ]
```

```
def filter_by_year(books: list[Book], year: int) -> list[Book]:
```

```
    return [book for book in books if book.year == year]
```

```
def find_largest_book(books: list[Book]) -> Book | None:
```

```
    if not books:
```

```
        return None
```

```
    return max(books, key=lambda book: book.pages)
```

```
def calculate_average_pages(books: list[Book]) -> float:
```

```
    if not books:
```

```
    return 0.0

total_pages = sum(book.pages for book in books)

return total_pages / len(books)
```

4.3. main.py

Модуль ініціалізує початковий набір даних, демонструє виклики функцій обробки, виконує форматоване табличне виведення та містить конструкцію `if __name__ == "__main__":`.

```
from book_manager.models import Book

from book_manager.services import (
    add_book,
    calculate_average_pages,
    filter_by_year,
    find_books_by_author,
    find_largest_book,
)

def create_demo_books() -> list[Book]:
    return [
        Book(title="The Hobbit", author="J.R.R. Tolkien", year=1937, pages=310),
        Book(title="1984", author="George Orwell", year=1949, pages=328),
        Book(title="Animal Farm", author="George Orwell", year=1945, pages=112),
        Book(title="The Lord of the Rings", author="J.R.R. Tolkien", year=1954, pages=1178),
        Book(title="Brave New World", author="Aldous Huxley", year=1932, pages=288),
    ]

def print_books(books: list[Book]) -> None:
    if not books:
        print("No books found.")
```

```

    return

print(f"{'Title':<25} {'Author':<20} {'Year':<6} {'Pages':<6}")

print("-" * 60)

for book in books:

    print(f"{book.title:<25} {book.author:<20} {book.year:<6} {book.pages:<6}")


def main() -> None:

    books = create_demo_books()

    print("All Books in Library:")

    print_books(books)


    new_book = Book(title="Fahrenheit 451", author="Ray Bradbury", year=1953,
pages=256)

    add_book(books, new_book)

    print(f"\nAdded new book: {new_book.short_info}")


    search_author = "Orwell"

    print(f"\nBooks by '{search_author}':")

    author_books = find_books_by_author(books, search_author)

    print_books(author_books)


    target_year = 1937

    print(f"\nBooks published in {target_year}:")

    year_books = filter_by_year(books, target_year)

    print_books(year_books)


    largest = find_largest_book(books)

    if largest:

        print(f"\nLargest book: {largest.short_info} with {largest.pages} pages")

```

```
avg_pages = calculate_average_pages(books)

print(f"Average pages across all books: {avg_pages:.1f}")
```

```
if __name__ == "__main__":

    main()
```

5. Файл pyproject.toml

Використовує стандартний бекенд `setuptools`, визначає метадані пакета, мінімальну версію Python 3.11 та реєструє точку запуску консольної утиліти `book-manager`.

```
[build-system]

requires = ["setuptools>=70"]

build-backend = "setuptools.build_meta"


[project]

name = "book-manager"

version = "0.1.0"

description = "Book management laboratory project"

requires-python = ">=3.11"


[project.scripts]

book-manager = "book_manager.main:main"
```

6. Файл .gitignore

Запобігає потраплянню до Git тимчасових системних файлів, віртуального середовища, конфігурацій IDE та артефактів збірки.

```
.venv/

__pycache__/

*.pyc
```

.pytest_cache/

.idea/

.vscode/

dist/

build/

***.egg-info/**

.env

7. Фрагмент README.md

Book Manager

Laboratory project for the course "Professional Python".

Description

Console application for managing books catalog, filtering, and calculating statistics.

Requirements

Python 3.11+

Installation

python -m venv .venv

.venv\Scripts\activate

python -m pip install -e .

Run

python -m book_manager.main

or

book-manager

8. Створення virtual environment та запуск

```
python -m venv .venv
```

```
.venv\Scripts\activate
```

```
python -m pip install -e .
```

```
python -m book_manager.main
```

9. Результат роботи програми

All Books in Library:

Title	Author	Year	Pages

The Hobbit	J.R.R. Tolkien	1937	310
1984	George Orwell	1949	328
Animal Farm	George Orwell	1945	112
The Lord of the Rings	J.R.R. Tolkien	1954	1178
Brave New World	Aldous Huxley	1932	288

Added new book: 'Fahrenheit 451' by Ray Bradbury (1953)

Books by 'Orwell':

Title	Author	Year	Pages

1984	George Orwell	1949	328
Animal Farm	George Orwell	1945	112

Books published in 1937:

Title	Author	Year	Pages

The Hobbit	J.R.R. Tolkien	1937	310

Largest book: 'The Lord of the Rings' by J.R.R. Tolkien (1954) with 1178 pages

Average pages across all books: 412.0

10. Робота з Git

Історія сформована з трьох логічних кроків відповідно до методичних вказівок:

git init

git add .gitignore pyproject.toml src/book_manager/__init__.py

git commit -m "Create project structure and configuration"

git add src/book_manager/models.py src/book_manager/services.py

git commit -m "Add Book model and business logic services"

git add src/book_manager/main.py README.md

git commit -m "Add main entry point and README documentation"

10.1. Результат git status

On branch main

nothing to commit, working tree clean

10.2. Результат git log --oneline

9ae4951 Add main entry point and README documentation

4f82bc1 Add Book model and business logic services

1b37a09 Create project structure and configuration

11. Відповіді на контрольні запитання

1. Що таке Python-проект?

Організована сукупність вихідного коду, конфігураційних файлів, тестів, документації та залежностей, що утворюють єдиний цілісний програмний продукт або бібліотеку.

2. Чим проект відрізняється від одного Python script?

Проект має модульну структуру, чітку точку входу, ізольоване оточення, керування залежностями через конфігураційні файли (наприклад, `pyproject.toml`), контроль версій та документацію.

3. Що таке Python module?

Окремий файл із розширенням `.py`, що містить функції, класи чи змінні для повторного використання.

4. Що таке Python package?

Каталог із модулями Python, що містить файл `__init__.py` (або є namespace package) і дозволяє організовувати модулі в ієрархію за допомогою крапкової нотації.

5. Для чого потрібний `__init__.py`?

Він позначає директорію як Python-пакет, виконує код його ініціалізації та експортує публічний інтерфейс через `__all__` або прямі імпорти.

6. Що таке src-layout?

Архітектурний підхід, за якого весь вихідний код пакетів розміщується у виділеній підпапці `src/`, а не в кореновому каталозі проекту.

7. Які переваги має src-layout?

Запобігає випадковому імпорту локального незібраного коду замість встановленого пакета під час тестування (проблема `import parity`), спрощує налаштування збирачів і тримає корінь проекту чистим.

8. Для чого використовується virtual environment?

Для повної ізоляції залежностей та версій сторонніх пакетів окремого проекту від загальносистемного інтерпретатора Python.

9. Як створити `.venv`?

Виконати команду: `python -m venv .venv`.

10. Як активувати virtual environment?

У Windows: `.venv\Scripts\activate` (або `.venv\Scripts\Activate.ps1`). У Linux/macOS: `source .venv/bin/activate`.

11. Для чого використовується `pip`?

Це стандартний пакетний менеджер для пошуку, завантаження, встановлення, оновлення та видалення пакетів з Python Package Index (PyPI) або локальних джерел.

12. Чому рекомендується `python -m pip`?

Ця конструкція гарантує запуск саме тієї версії `pip`, яка прив'язана до конкретного запущеного інтерпретатора Python (зокрема всередині активного віртуального оточення), уникаючи колізій із системним шляхом `PATH`.

13. Для чого потрібний pyproject.toml?

Це стандартизований (PEP 518/621) декларативний файл конфігурації для визначення інструментів збірки (build-backend), метаданих проєкту, залежностей, скриптів CLI та параметрів linters/formatters/тестів.

14. Що містить секція [project]?

Основні метадані: назву проєкту (name), версію (version), опис (description), сумісні версії Python (requires-python), авторів, залежності та точки входу (project.scripts).

15. Що таке editable installation?

Режим встановлення пакета (pip install -e .), при якому в середовище монтуються посилання на вихідний код у src/. Будь-які зміни в файлах коду стають доступними миттєво без потреби повторного виконання збірки й встановлення.

16. Для чого використовується if __name__ == "__main__":?

Щоб блок коду запускався виключно тоді, коли файл викликається як головна програма, і не виконувався при імпорті модуля в інші файли чи тести.

17. Що таке Git?

Розподілена система керування версіями файлів, яка дозволяє фіксувати зміни коду, відкочуватися до попередніх станів і вести паралельну розробку у гілках.

18. Що таке repository?

Структура даних (прихована папка .git), що містить усю історію комітів, гілок, тегів та конфігурацію відстеження файлів проєкту.

19. Що таке commit?

Знімок стану проєкту (снэпшот) у певний момент часу, що має унікальний SHA-хеш, метадані про автора, дату створення та коментар із описом внесених змін.

20. Для чого використовується git status?

Для перегляду поточного стану робочої директорії та індексу (staging area): показує змінені, підготовлені до коміту чи невідстежувані файли.

21. Для чого git add?

Для перенесення нових або модифікованих файлів із робочого каталогу до staging area (індексу) для подальшої фіксації.

22. Для чого git commit?

Для збереження підготовлених у staging area змін до локальної бази даних репозиторію у вигляді нового коміту.

23. Що показує git log?

Хронологічний список створених комітів у поточній гілці із зазначенням хешу, автора, дати та опису.

24. Для чого потрібен .gitignore?

Щоб вказати патерни файлів і папок, які Git повинен ігнорувати і ніколи не додавати до відстеження репозиторію.

25. Чому .venv не потрібно комітити?

Каталог містить бінарні платформозалежні файли інтерпретатора конкретної ОС, має великий розмір і легко відтворюється будь-яким розробником на основі метаданих проєкту командою встановлення.

26. Для чого потрібний README?

Для документування проєкту: він містить опис функціональності, системні вимоги, інструкції зі встановлення, налаштування середовища та приклади запуску.

27. Що таке type hints?

Синтаксичні анотації очікуваних типів змінних, аргументів функцій та значень, що повертаються. Вони підвищують читабельність коду, покращують автодоповнення в IDE та дозволяють проводити статичний аналіз (наприклад, через туру).

28. Для чого програму розділяють на modules?

Для дотримання принципу розділення відповідальності (Separation of Concerns), зменшення зв'язності коду, спрощення повторного використання компонентів та ізольованого тестування.

29. Що таке business logic?

Набір алгоритмів, правил та обчислень предметної області, які вирішують прикладну задачу і не залежать від способу введення даних чи графічного/консольного інтерфейсу (наприклад, фільтрація книг або підрахунок середньої кількості сторінок).

30. Чому потрібно відокремлювати application entry point від business logic?

Щоб функції обробки даних можна було тестувати без запуску всієї програми, перевикористовувати в інших інтерфейсах (наприклад, веб-API чи GUI) та уникати сайд-ефектів від операцій консольного вводу-виводу.

12. Висновок

У ході виконання лабораторної роботи було створено структурований консольний Python-застосунок за принципом src-layout відповідно до вимог варіанта №2 («Система обліку книг»). Реалізовано дата-клас Book, окремий модуль бізнес-логіки з функціями фільтрації, пошуку та агрегації, а також точку входу main.py з повним покриттям type hints. Створено та налаштовано файли pyproject.toml, .gitignore і README.md. Пакет успішно встановлено у віртуальне середовище в editable-режимі. За допомогою системи

Git зафіксовано історію з трьох логічних комітів та успішно пройдено перевірку на платформі автоматизованого тестування.